Project 18.1 — LessonForge-Topic-To-Cited-Lesson-Plan-Builder

A practice twin of Project 18 — ResearchPilot. Same skills — a plan-first agent that breaks a topic into sub-parts, researches each against your documents, and compiles a report with **citations** — on a new real-world job. **There are no steps here on purpose.** Read the story, build it yourself, and open the hints or solution only if you get stuck.

Type: Planner-executor agent (RAG) · **Stack:** Python, LangChain, Chroma (RAG), OpenAI SDK, Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A teacher (or a self-learner) wants a lesson on a topic — "teach me vectors" — and a one-shot answer from a chatbot is a wall of text with no structure and no sources. A good lesson is *planned*: broken into sub-topics, each explained from trustworthy material, with pointers back to where to read more.

You are going to build **LessonForge**: give it a topic, get back a structured, cited lesson plan.

- A **planner** turns the topic into a short list of sub-topics (the lesson's sections).
- An **executor** searches your content library for each sub-topic and collects an explanation (the retrieved text + its source).
- A **compiler** assembles the sections into a lesson with numbered **citations**, so each part points back to the document it came from.

The one lesson of this project: same as ResearchPilot — a planned, cited output is **plan → research each part → compile**, and citations are what make it trustworthy. The executor takes a `search` function (not a hard-wired store), so the plan-and-compile engine is testable with a fake search, no model or embeddings.

How it works, from a topic to a cited lesson (picture the whole machine)

A topic comes in. The **planner** (a structured LLM call) breaks it into sub-topics. The **executor** runs each sub-topic through a **search** function — a RAG lookup over your content library — and collects a `Section` (the retrieved explanation plus its source). A deterministic **compiler** folds the sections into a `Lesson`, and a **formatter** prints it with numbered citations, de-duplicating sources so the same document is `[1]` throughout.

That's the whole machine: **topic → plan sub-topics → research each → cited lesson plan.**

What you are building (the pieces)

- Typed shapes: `SubTopics(sub_topics)`, `Section(sub_topic, explanation, source)`, `Lesson(topic, sections)`.
- A deterministic `build_sections(sub_topics, search)` + `compile_lesson` + `format_lesson` (with citations).
- A RAG layer over the content library (`build_library_store`, `make_search`).
- A planner (`make_plan`) and a pipeline (`run_lesson`).

Inputs: a topic. **Outputs:** a cited lesson plan. **Rules:** plan first; each section grounded in a retrieved document with its source; citations numbered and de-duplicated; the engine takes a `search` function (testable with no model).

A real example to build and test against

- Topic "retrieval-augmented generation" → sections like "What is RAG?", "How does chunking work?", each explained from your docs and cited `[1]`, `[2]`.
- Two sub-topics whose explanations come from the same file → both cite `[1]`, and the Sources list shows that file once.

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 18 / Doc07 plan-first / Doc08 RAG if hazy.)
2. Build the **engine first** — `build_sections` + `compile_lesson` + `format_lesson` — and test the citation logic with a fake `search`, no model.
3. Build the planner (structured) and the RAG search (library store + a `search(q) -> (text, source)` closure).
4. Wire them: plan → search → build sections → compile.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

"Plan a topic into parts, pull trustworthy content for each, and cite it" is the shape of lesson builders, study guides, onboarding docs, and research briefings alike. The skill is the structure — plan first, ground each part, cite the source — and it's reusable anywhere an answer must be organized and verifiable. Build LessonForge and you can turn any topic plus a document library into a structured, cited output.

Done when

- The topic is planned into sub-topics *before* any search runs.
- Each section is grounded in a retrieved document, with its source.
- Every section is cited; sources are numbered and de-duplicated.
- `build_sections` takes a `search` function, so the engine tests with no model.
- Your tests pass: the engine (fake search) and a live lesson on a real topic.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.